

tcpdump: The 80/20 Guide for Kerberos Security

Master the 20% That Gives You 80% of the Value

Author: Emerson

Focus: Kerberos traffic capture for security analysis

Time to Competence: 30 minutes

Core Concept

tcpdump captures network packets. For your Kerberos project, you need to:

1. Capture Kerberos traffic (port 88)
2. Save to pcap files
3. Analyze with Wireshark

That's it. Everything else is bonus.

The Essential 20%

Command Structure (Only This Matters)

```
tcpdump [options] [filter] -w output.pcap
```

3 parts:

- **Options:** How to capture
 - **Filter:** What to capture
 - **Output:** Where to save
-

Quick Start: Your First Capture

Capture 1: All Kerberos Traffic

```
sudo tcpdump -i any port 88 -w kerberos_capture.pcap
```

What this does:

- `-i any` = Listen on all network interfaces
- `port 88` = Only Kerberos traffic
- `-w file.pcap` = Save to file

When to use: General Kerberos monitoring

Stop it: Ctrl+C

Capture 2: Kerberos from Specific IP

```
sudo tcpdump -i any port 88 and host 192.168.88.10 -w attack_192.168.88.10.pcap
```

What this does:

- Only captures Kerberos traffic from/to 192.168.88.10

When to use: Isolating attacker traffic

Capture 3: Kerberos with Packet Details

```
sudo tcpdump -i any -nn -vv port 88 -w detailed_capture.pcap
```

Options explained:

- `-nn` = Don't resolve hostnames/ports (faster, clearer)
- `-vv` = Very verbose (more packet details)
- `-w` = Save to file

When to use: Detailed analysis needed

The 6 Essential Filters

These cover 95% of Kerberos security work:

1. Capture Kerberos Only

```
sudo tcpdump -i any port 88 -w kerberos.pcap
```

2. Capture from Specific IP

```
sudo tcpdump -i any port 88 and host 192.168.88.10 -w attacker.pcap
```

3. Capture Kerberos + DNS (for troubleshooting)

```
sudo tcpdump -i any 'port 88 or port 53' -w kerb_and_dns.pcap
```

4. Capture Between Two Hosts

```
sudo tcpdump -i any port 88 and host 192.168.88.10 and host 192.168.88.20 -w client_to_k
```

5. Exclude Your Own IP (avoid noise)

```
sudo tcpdump -i any port 88 and not host 192.168.88.254 -w no_monitoring.pcap
```

6. Capture with Timestamps

```
sudo tcpdump -i any -tttt port 88 -w timestamped.pcap
```

- `-tttt` = Human-readable timestamps
-



Real Kerberos Attack Scenarios

Scenario 1: Capture Brute Force Attack

Setup:

```
# Terminal 1: Start capture
sudo tcpdump -i any port 88 -w bruteforce_capture.pcap

# Terminal 2: Run brute force attack
python3 01_kerb_bruteforce_advanced.py -r LAB.LOCAL -k 192.168.88.20 -u admin -p password

# Terminal 1: Stop capture (Ctrl+C) after attack finishes
```

What you'll have:

- Complete packet capture of brute force
 - Every authentication attempt
 - Failed/successful responses
 - Perfect for analysis
-

Scenario 2: Capture AS-REP Roasting

Setup:

```
# Start capture
sudo tcpdump -i any port 88 -w asrep_roasting.pcap

# In another terminal, run AS-REP roast
python3 02_asrep_roast_automated.py -r LAB.LOCAL -k 192.168.88.20

# Stop capture when done
```

What you'll see:

- AS-REQ without pre-auth
 - AS-REP with encrypted hash
 - User enumeration patterns
-

Scenario 3: Capture Kerberoasting

Setup:

```
sudo tcpdump -i any port 88 -w kerberoasting.pcap

# Run Kerberoasting
python3 03_kerberoast_intelligent.py -r LAB.LOCAL -k 192.168.88.20

# Stop capture
```

What you'll see:

- Multiple TGS-REQ for service tickets
 - Different service principals
 - Ticket responses
-

Scenario 4: Capture with Your Monitoring System

Integrated approach:

```
# Terminal 1: Start capture
sudo tcpdump -i any port 88 -w performance_test.pcap

# Terminal 2: Start monitoring
sudo python3 kerberos_performance_monitor.py --once

# Terminal 3: Create chaos
python3 kerberos_chaos_toolkit.py cpu_exhaustion --duration 60

# Stop capture after chaos ends
```

What you'll have:

- Network data (pcap)
 - System metrics (monitoring)
 - Correlated timeline
-

Useful Options Reference

Display Options

```
# Verbose output (see more details)
-v          # Verbose
-vv         # Very verbose
-vvv        # Maximum verbosity

# Timestamps
-t          # No timestamp
-tt         # Unix timestamp
-ttt        # Time since previous packet
-tttt       # Human-readable date + time (BEST for analysis)

# Don't resolve names (faster, clearer)
-n          # Don't resolve hostnames
-nn         # Don't resolve hostnames or ports
```

Capture Options

```
# Interface
-i eth0     # Specific interface
-i any      # All interfaces (BEST for lab)

# Packet size
-s 0        # Capture full packets (default since 2009)
-s 1500     # Capture first 1500 bytes

# Count
-c 100      # Capture only 100 packets

# Buffer
-B 4096     # Increase buffer size (prevent drops)
```



Pro Tips for Your Project

Tip 1: Create Capture Scripts

brute_force_capture.sh:

```
#!/bin/bash
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
OUTFILE="captures/bruteforce_${TIMESTAMP}.pcap"

echo "Starting capture: $OUTFILE"
echo "Press Ctrl+C to stop"

sudo tcpdump -i any -nn -tttt port 88 -w "$OUTFILE"

echo "Capture saved: $OUTFILE"
echo "Packets captured: $(tcpdump -r "$OUTFILE" | wc -l)"
```

Usage:

```
chmod +x brute_force_capture.sh
./brute_force_capture.sh
```

Tip 2: Verify Capture Quality

After capturing, check it:

```
# Count packets
tcpdump -r capture.pcap | wc -l

# Quick preview (first 10 packets)
tcpdump -r capture.pcap -c 10

# Check for drops
tcpdump -r capture.pcap -vv | grep "packets dropped"
```

Tip 3: Capture While Monitoring Disk Space

For long captures:

```
# Rotate captures every 100MB  
sudo tcpdump -i any port 88 -w capture.pcap -C 100 -W 5
```

- `-C 100` = New file every 100MB
 - `-W 5` = Keep only last 5 files
 - Creates: capture.pcap0, capture.pcap1, etc.
-

Tip 4: Live Monitoring While Capturing

See packets in real-time AND save:

```
sudo tcpdump -i any port 88 -w capture.pcap -l | tee capture.log
```

- Saves to capture.pcap
 - Displays on screen
 - Logs to capture.log
-



Cheat Sheet: Common Kerberos Filters

```
# Basic Kerberos  
port 88  
  
# Specific KDC  
port 88 and host 192.168.88.20  
  
# Attacker IP  
port 88 and src 192.168.88.10  
  
# Traffic TO KDC  
port 88 and dst 192.168.88.20  
  
# Traffic FROM KDC  
port 88 and src 192.168.88.20
```



```
# Multiple IPs
port 88 and (host 192.168.88.10 or host 192.168.88.11)

# Exclude monitoring VM
port 88 and not host 192.168.88.254

# Kerberos + DNS
port 88 or port 53

# Large packets only (possible attacks)
port 88 and greater 1000

# Failed auth indicators (see in Wireshark)
port 88 # Then analyze error codes in Wireshark
```

Your Capture Workflow

For Each Attack Scenario:

1. Prepare:

```
mkdir -p ~/kerberos-captures/$(date +%Y%m%d)
cd ~/kerberos-captures/$(date +%Y%m%d)
```

2. Start Capture:

```
sudo tcpdump -i any -nn -tttt port 88 -w attack_name.pcap
```

3. Run Attack:

```
# In another terminal
python3 your_attack_script.py [options]
```

4. Stop Capture:

```
# Ctrl+C in tcpdump terminal
```

5. Verify:

```
tcpdump -r attack_name.pcap | wc -l  
echo "Captured $(tcpdump -r attack_name.pcap | wc -l) packets"
```

6. Quick Preview:

```
tcpdump -r attack_name.pcap -c 5 -vv
```

7. Open in Wireshark:

```
wireshark attack_name.pcap &  
# Or copy to your workstation
```



Troubleshooting

Problem: "Permission denied"

Solution:

```
# Use sudo  
sudo tcpdump ...  
  
# Or add your user to capture group  
sudo usermod -aG wireshark $USER  
# Log out and back in
```

Problem: "No packets captured"

Check:

```
# 1. Verify interface exists  
ip link show  
  
# 2. Check if port 88 has traffic  
sudo netstat -tulpn | grep :88
```

```
# 3. Try without filter first
sudo tcpdump -i any -c 10
```

Problem: "Dropped packets"

Solution:

```
# Increase buffer
sudo tcpdump -i any -B 16384 port 88 -w capture.pcap

# Or reduce verbosity
sudo tcpdump -i any port 88 -w capture.pcap
# (Don't use -vv during capture)
```

File Organization

Recommended structure:

```
kerberos-captures/
├── 20250207/
│   ├── bruteforce_admin_101530.pcap
│   ├── bruteforce_admin_101530.log
│   ├── asrep_roast_103245.pcap
│   ├── kerberoasting_111420.pcap
│   └── README.md (notes about captures)
├── 20250208/
└── scripts/
    ├── capture_bruteforce.sh
    ├── capture_asrep.sh
    └── capture_kerberoast.sh
```

Quick Reference Card

Print this and tape to your monitor:

tcpdump Quick Reference

BASIC CAPTURE:

```
sudo tcpdump -i any port 88 -w file.pcap
```

WITH DETAILS:

```
sudo tcpdump -i any -nn -tttt port 88 -w file.pcap
```

SPECIFIC HOST:

```
sudo tcpdump -i any port 88 and host IP -w file.pcap
```

READ CAPTURE:

```
tcpdump -r file.pcap
```

COUNT PACKETS:

```
tcpdump -r file.pcap | wc -l
```

STOP CAPTURE:

```
Ctrl+C
```

Practice Exercises

Exercise 1: Basic Capture

```
# Capture 100 Kerberos packets
sudo tcpdump -i any port 88 -c 100 -w test.pcap

# Verify
tcpdump -r test.pcap | wc -l

# Should show: 100
```

Exercise 2: Filtered Capture

```
# Capture only from attacker VM
sudo tcpdump -i any port 88 and host 192.168.88.10 -w attacker.pcap

# Generate traffic from that VM
ssh 192.168.88.10 "kinit testuser@LAB.LOCAL"

# Check capture
tcpdump -r attacker.pcap
```

Exercise 3: Your First Attack Capture

```
# Capture a full brute force attempt
sudo tcpdump -i any port 88 -w my_first_attack.pcap

# Run brute force with 10 passwords
python3 01_bruteforce.py -r LAB.LOCAL -k 192.168.88.20 -u admin -p short_passwords.txt

# Analyze in Wireshark (next guide!)
```



Additional Resources

Man page (detailed reference):

```
man tcpdump
```

Online resources:

- tcpdump.org - Official site
- [PCAP filter syntax](#) - Filter reference



Mastery Checklist

You've mastered tcpdump when you can:

- [] Capture Kerberos traffic to a file

- [] Filter by IP address
- [] Read and count packets from a capture
- [] Create named captures for each attack type
- [] Verify capture quality (no drops)
- [] Integrate with your attack scripts
- [] Organize captures in dated directories

That's it. That's the 20% that matters.

Next Steps

1. **Practice:** Run each example once
 2. **Create:** Make your capture scripts
 3. **Organize:** Set up your directory structure
 4. **Capture:** Record one of each attack type
 5. **Next:** Learn Wireshark to analyze them
-

You're now competent in tcpdump for Kerberos security work.

Next guide: Wireshark analysis 👉

Created by: Emerson

Part of: Enterprise Kerberos Security Project

Time to Read: 15 minutes

Time to Competence: 30 minutes practice